ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №3

Студент Трофимов А. В.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

САМАРА 2025

Задание 2

Созданы два специализированных класса исключений: FunctionPointIndexOutOfBoundsException для обработки неверных индексов и InappropriateFunctionPointException для нарушений упорядоченности точек.

functions > FunctionPointIndexOutOfBoundsException.java > Language Support for Java(TM) by Red Hat > FunctionPoint

```java
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
    public FunctionPointIndexOutOfBoundsException() {}
    public FunctionPointIndexOutOfBoundsException(String s) { super(s); }
}
```

functions > InappropriateFunctionPointException.java > Language Support for Java(TM) by Red Hat >

```java
package functions;

public class InappropriateFunctionPointException extends Exception {
    public InappropriateFunctionPointException() {}
    public InappropriateFunctionPointException(String s) { super(s); }
}
```

Задание 3

Доработан класс TabulatedFunction (переименованный позже в ArrayTabulatedFunction) с добавлением проверок и выбрасыванием исключений при некорректных операциях.

```java
package functions;

public class ArrayTabulatedFunction implements TabulatedFunction {
    private FunctionPoint[] points;
    private int pointsCount;

    public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
        if (leftX >= rightX || pointsCount < 2)
            throw new IllegalArgumentException(s: "Invalid domain or points count");

        this.points = new FunctionPoint[pointsCount + 10];
        this.pointsCount = pointsCount;
        double step = (rightX - leftX) / (pointsCount - 1);

        for (int i = 0; i < pointsCount; i++) {
            double x = leftX + i * step;
            points[i] = new FunctionPoint(x, y: 0);
        }
    }

    public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
        if (leftX >= rightX || values.length < 2)
            throw new IllegalArgumentException(s: "Invalid domain or points count");

        this.points = new FunctionPoint[values.length + 10];
        this.pointsCount = values.length;
        double step = (rightX - leftX) / (values.length - 1);

        for (int i = 0; i < values.length; i++) {
            double x = leftX + i * step;
            points[i] = new FunctionPoint(x, values[i]);
        }
    }
```

```java
    @Override
    public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
        checkIndex(index);

        if ((index > 0 && point.getX() <= points[index - 1].getX()) ||
            (index < pointsCount - 1 && point.getX() >= points[index + 1].getX())) {
            throw new InappropriateFunctionPointException(s: "Invalid X order");
        }

        points[index] = new FunctionPoint(point);
    }

    @Override
    public double getPointX(int index) {
        checkIndex(index);
        return points[index].getX();
    }

    @Override
    public void setPointX(int index, double x) throws InappropriateFunctionPointException {
        checkIndex(index);

        if ((index > 0 && x <= points[index - 1].getX()) ||
            (index < pointsCount - 1 && x >= points[index + 1].getX())) {
            throw new InappropriateFunctionPointException(s: "Invalid X order");
        }

        points[index].setX(x);
    }
```

Задание 4

Создан класс LinkedListTabulatedFunction с двусвязным циклическим списком для хранения точек, включая внутренний класс Node и методы работы со списком.

```
1   package functions;
2
3   public class LinkedListTabulatedFunction implements TabulatedFunction {
4
5       private class Node {
6           FunctionPoint point;
7           Node prev, next;
8           Node(FunctionPoint p) { point = p; }
9       }
```

```
52      private Node getNode(int index) {
53          if (index < 0 || index >= size)
54              throw new FunctionPointIndexOutOfBoundsException("Index: " + index);
55
56          Node current = head.next;
57          for (int i = 0; i < index; i++)
58              current = current.next;
59
60          return current;
61      }
```

Задание 5

Класс LinkedListTabulatedFunction дополнен методами интерфейса, обеспечивающими работу с табулированной функцией, включая конструкторы, методы доступа к точкам и вычисление значений.

```
14      public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
15          if (leftX >= rightX || pointsCount < 2)
16              throw new IllegalArgumentException(s: "Invalid domain or points count");
17
18          init();
19          double step = (rightX - leftX) / (pointsCount - 1);
20          for (int i = 0; i < pointsCount; i++)
21              addNodeToTail(new FunctionPoint(leftX + i * step, y: 0));
22      }
23
24      public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
25          if (leftX >= rightX || values.length < 2)
26              throw new IllegalArgumentException(s: "Invalid domain or points count");
27
28          init();
29          double step = (rightX - leftX) / (values.length - 1);
30          for (int i = 0; i < values.length; i++)
31              addNodeToTail(new FunctionPoint(leftX + i * step, values[i]));
32      }
```

```
93      @Override
94      public double getFunctionValue(double x) {
95          if (x < getLeftDomainBorder() || x > getRightDomainBorder())
96              return Double.NaN;
97
98          Node current = head.next;
99          for (int i = 0; i < size; i++) {
100             if (Math.abs(current.point.getX() - x) < 1e-9)
101                 return current.point.getY();
102             current = current.next;
103         }
104
105         current = head.next;
106         for (int i = 0; i < size - 1; i++) {
107             if (x >= current.point.getX() && x <= current.next.point.getX()) {
108                 double x1 = current.point.getX(), y1 = current.point.getY();
109                 double x2 = current.next.point.getX(), y2 = current.next.point.getY();
110                 return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
111             }
112             current = current.next;
113         }
114         return Double.NaN;
115     }
```

Задание 6

Создан интерфейс TabulatedFunction и переименован TabulatedFunction в ArrayTabulatedFunction, оба класса теперь реализуют единый интерфейс.

```java
package functions;

public interface TabulatedFunction {
    double getLeftDomainBorder();
    double getRightDomainBorder();
    double getFunctionValue(double x);

    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point)throws InappropriateFunctionPointException, FunctionPointIndexOutOfBoundsException;
    double getPointX(int index) throws FunctionPointIndexOutOfBoundsException;
    void setPointX(int index, double x) throws InappropriateFunctionPointException, FunctionPointIndexOutOfBoundsException;
    double getPointY(int index) throws FunctionPointIndexOutOfBoundsException;
    void setPointY(int index, double y) throws FunctionPointIndexOutOfBoundsException;

    void deletePoint(int index) throws FunctionPointIndexOutOfBoundsException, IllegalStateException;
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
}
```

```java
public class ArrayTabulatedFunction implements TabulatedFunction {
```

```java
public class LinkedListTabulatedFunction implements TabulatedFunction {
```

Задание 7

Изменена тестовая программа Main, подтверждающая корректную работу обоих классов, обработку исключений и взаимозаменяемость реализаций через интерфейс.

```java
import functions.*;

public class Main {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        double[] data = {0,1,4,9,16};
        // Тестируем ArrayTabulatedFunction
        System.out.println(x: "=== Тестирование ArrayTabulatedFunction ===");
        testFunction(new ArrayTabulatedFunction(leftX: 0, rightX: 4, data));

        // Тестируем LinkedListTabulatedFunction
        System.out.println(x: "\n\n=== Тестирование LinkedListTabulatedFunction ===");
        testFunction(new LinkedListTabulatedFunction(leftX: 0, rightX: 4, data));
    }

    private static void testFunction(TabulatedFunction function) {
        System.out.println(x: "\nИсходная функция:");
        printFunctionInfo(function);

        // Проверяем значения функции в разных точках
        System.out.println(x: "\nЗначения функции в разных точках:");
        double[] testPoints = {-1, 0, 0.5, 1, 1.5, 2, 3, 4, 5};
        for (double x : testPoints) {
            double y = function.getFunctionValue(x);
            System.out.printf(format: "f(%.1f) = %s\n", x, Double.isNaN(y) ? "NaN" : String.format(format: "%.2f", y));
        }

        // Изменяем точки
        System.out.println(x: "\nПосле изменения точек:");
        function.setPointY(index: 2, y: 5);
        try {
            function.setPoint(index: 3, new FunctionPoint(x: 3.5, y: 12.3));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Ошибка при setPoint: " + e.getMessage());
        }
        printFunctionInfo(function);

        // Добавляем точку
        System.out.println(x: "\nПосле добавления точки (2.5, 6.3):");
        try {
            function.addPoint(new FunctionPoint(x: 2.5, y: 6.3));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Ошибка при addPoint: " + e.getMessage());
        }
        printFunctionInfo(function);

        // Удаляем точку
        System.out.println(x: "\nПосле удаления точки с индексом 1:");
        try {
            function.deletePoint(index: 1);
        } catch (Exception e) {
            System.out.println("Ошибка при deletePoint: " + e.getClass().getSimpleName() + " - " + e.getMessage());
        }
        printFunctionInfo(function);
    }

    private static void printFunctionInfo(TabulatedFunction function) {
        System.out.printf(format: "Область определения: [%.1f, %.1f]\n",
            function.getLeftDomainBorder(), function.getRightDomainBorder());
        System.out.println(x: "Точки функции:");
        for (int i = 0; i < function.getPointsCount(); i++) {
            FunctionPoint point = function.getPoint(i);
            System.out.printf(format: "  (%4.1f; %6.2f)\n", point.getX(), point.getY());
        }
    }
}
```

Реализация:

```
=== Тестирование ArrayTabulatedFunction ===

Исходная функция:
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 1,0;   1,00)
  ( 2,0;   4,00)
  ( 3,0;   9,00)
  ( 4,0;  16,00)

Значения функции в разных точках:
f(-1,0) = NaN
f(0,0) = 0,00
f(0,5) = 0,50
f(1,0) = 1,00
f(1,5) = 2,50
f(2,0) = 4,00
f(3,0) = 9,00
f(4,0) = 16,00
f(5,0) = NaN

После изменения точек:
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 1,0;   1,00)
  ( 2,0;   5,00)
  ( 3,5;  12,30)
  ( 4,0;  16,00)

После добавления точки (2.5, 6.3):
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 1,0;   1,00)
  ( 2,0;   5,00)
  ( 2,5;   6,30)
  ( 3,5;  12,30)
  ( 4,0;  16,00)

После удаления точки с индексом 1:
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 2,0;   5,00)
  ( 2,5;   6,30)
  ( 3,5;  12,30)
  ( 4,0;  16,00)
```

```
--- Тестирование LinkedListTabulatedFunction ---

Исходная функция:
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 1,0;   1,00)
  ( 2,0;   4,00)
  ( 3,0;   9,00)
  ( 4,0;  16,00)

Значения функции в разных точках:
f(-1,0) = NaN
f(0,0) = 0,00
f(0,5) = 0,50
f(1,0) = 1,00
f(1,5) = 2,50
f(2,0) = 4,00
f(3,0) = 9,00
f(4,0) = 16,00
f(5,0) = NaN

После изменения точек:
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 1,0;   1,00)
  ( 2,0;   5,00)
  ( 3,5;  12,30)
  ( 4,0;  16,00)

После добавления точки (2.5, 6.3):
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 1,0;   1,00)
  ( 2,0;   5,00)
  ( 2,5;   6,30)
  ( 3,5;  12,30)
  ( 4,0;  16,00)

После удаления точки с индексом 1:
Область определения: [0,0, 4,0]
Точки функции:
  ( 0,0;   0,00)
  ( 2,0;   5,00)
  ( 2,5;   6,30)
  ( 3,5;  12,30)
  ( 4,0;  16,00)
```